

What You'll Build

A WhatsApp-style chat screen inside a booking. The chat is **only open during the session window**:

The Chat Window — Read This Carefully

- **Opens 5 minutes BEFORE** the scheduled start time → patient/healer can arrive early (waiting room)
- **Closes EXACTLY AT** the scheduled end time → **no buffer, no grace period**
- **Example:** Session booked 3:00 PM – 3:30 PM
 - `chatOpensAt` = **2:55 PM**
 - `chatClosesAt` = **3:30 PM** (= `ScheduledEndAt`)
- Outside this window: **history is still readable**, but no new messages can be sent
- To chat again, the patient must **book a new session**

Recommended Flutter Packages

pubspec.yaml

```
dependencies:
  signalr_netcore: ^1.4.0      # SignalR client
  dio: ^5.4.0                  # REST + interceptor for JWT
  flutter_secure_storage: ^9.0.0 # Store JWT
  intl: ^0.19.0                # Date/time formatting
  provider: ^6.1.1             # Or riverpod / bloc — your choice
  uuid: ^4.2.1                 # For idempotencyKey on send
```

The Scenario We'll Walk Through

Patient Aarav booked a 30-min chat session with **Healer Dr. Riya** for today at **3:00 PM – 3:30 PM**.

This means:

- **chatOpensAt** = 2:55 PM (5 min before start)
- **scheduledStartAt** = 3:00 PM
- **scheduledEndAt** = 3:30 PM
- **chatClosesAt** = 3:30 PM (same as end — no buffer)

2:45 PM — Aarav Opens His Bookings List

Step 1 — Check chat access

Call: `GET /api/v1/chat/access/{bookingId}` with Bearer JWT

Response (10 min before opens):

```
{
  "canRead": true,
  "canWrite": false,
  "code": "SESSION_NOT_STARTED",
  "message": "Chat opens at 2:55 PM...",
  "chatOpensAt": "2026-06-01T14:55:00Z",
  "chatClosesAt": "2026-06-01T15:30:00Z",
  "scheduledStartAt": "2026-06-01T15:00:00Z",
  "scheduledEndAt": "2026-06-01T15:30:00Z",
  "serverTimeUtc": "2026-06-01T14:45:00Z"
}
```

Flutter shows:

- A countdown "Chat opens in 10m 00s" driven by `chatOpensAt - serverTimeUtc`
- Input bar is **disabled**
- **Do not** open the SignalR connection yet — wait until `canWrite` is true
- Save `clockDrift = DateTime.now().toUtc() - serverTimeUtc` — use this for all countdowns so phone-clock skew doesn't matter
- Notice: `chatClosesAt == scheduledEndAt` — they're identical now

Flutter snippet:

```
final response = await dio.get('/chat/access/$bookingId');
final access = ChatAccess.fromJson(response.data);

if (access.canWrite) {
  // Connect SignalR + open chat screen
  await chatService.connect(bookingId);
} else if (access.code == 'SESSION_NOT_STARTED') {
  showWaitingRoom(opensAt: access.chatOpensAt);
} else if (access.code == 'SESSION_ENDED') {
  showHistoryOnly(showBookAgainCta: true);
}
```

2:56 PM — Window Just Opened

Step 2 — Re-check access

Same call. Now returns `canWrite: true, code: "OK"`. Proceed to load history.

Step 3 — Load message history

Call: `GET /api/v1/chat/history/{bookingId}?page=1&pageSize=50`

📎 **Response (empty on first visit):**

```
{
  "conversationId": "...",
  "messages": [],
  "totalCount": 0,
  "unreadCount": 0,
  "hasMore": false,
  "page": 1,
  "pageSize": 50
}
```

Step 4 — Connect to SignalR

```
final hubConnection = HubConnectionBuilder()
  .withUrl(
    'https://api.healer.com/hubs/chat',
    options: HttpConnectionOptions(
      accessTokenFactory: () async => jwtToken,
    ),
  )
  .withAutomaticReconnect()
  .build();

await hubConnection.start();
```

Step 5 — Join the chat room

```
await hubConnection.invoke('JoinChat', args: [bookingId]);
```

Server checks: ☒ Is Aarav a participant? ☒ Is session in window? **Yes** → joins room.

📎 **Server fires `ChatWindow` event:**

```
{
  "bookingId": "...",
  "chatOpensAt": "2026-06-01T14:55:00Z",
  "chatClosesAt": "2026-06-01T15:30:00Z",
  "scheduledStartAt": "2026-06-01T15:00:00Z",
  "scheduledEndAt": "2026-06-01T15:30:00Z"
}
```

📱 **Flutter:** Start a local `Timer` that fires at `chatClosesAt = 3:30 PM` → auto-disable composer at that moment **before** the next send fails server-side.

📎 **Because Aarav is FIRST to join — server sends 2 system messages:**

"Welcome, Aarav 🙌"

"Your session with Dr. Riya is scheduled at 03:00 PM. The healer will join shortly — please wait here."

These arrive as `MessageSent` events with `messageType: "System"`.

```
hubConnection.on('MessageSent', (args) {
  final msg = ChatMessage.fromJson(args![0]);

  if (msg.messageType == 'System') {
    // Render as centered grey pill, no avatar, italic
    chatList.add(SystemMessageBubble(text: msg.content));
  } else {
    chatList.add(MessageBubble(message: msg));
  }
});
```

3:00 PM — Dr. Riya Joins

She runs the same Steps 1-5. When she calls `JoinChat`, server sees she's the **second** to arrive.

 **Broadcast to everyone in the room:**

"Dr. Riya has joined the session"

Aarav also gets `UserOnline` event → show **green dot** next to Riya's name in the header.

```
hubConnection.on('UserOnline', (args) {
  final data = args![0];
  setState(() => onlineUsers.add(data['userId']));
});

hubConnection.on('UserOffline', (args) {
  final data = args![0];
  setState(() => onlineUsers.remove(data['userId']));
});
```

3:01 PM — Aarav Sends a Message

Step 6 — Show "typing" indicator

```
// On first keystroke in TextField:
await hubConnection.invoke('StartTyping', args: [bookingId]);

// After 3 seconds of no typing (debounce):
await hubConnection.invoke('StopTyping', args: [bookingId]);
```

Riya's app receives `TypingStarted` → shows "Aarav is typing...".

Step 7 — Send the message

```
final idempotencyKey = Uuid().v4(); // CRITICAL - prevents dupes on retry
await hubConnection.invoke('SendMessage', args: [
  bookingId,
  "Hi doctor, I've been feeling anxious",
  "Text",
  idempotencyKey,
  null, // attachmentJson (null for text)
]);
```

📡 Server broadcasts to room:

```
{
  "messageId": "msg-001",
  "senderId": "aarav-id",
  "senderName": "Aarav",
  "senderRole": "Patient",
  "content": "Hi doctor, I've been feeling anxious",
  "messageType": "Text",
  "createdAt": "2026-06-01T15:01:30Z",
  "readBy": [],
  "isDelivered": true
}
```

📱 **Flutter pattern:** Use **optimistic UI** — add the message locally *before* the invoke completes (with a "sending" spinner). When `MessageSent` arrives, replace it. Match by `idempotencyKey`.

Step 8 — Read receipts

When Riya opens the chat, the message appears on her screen → Flutter calls:

```
await hubConnection.invoke('MarkRead', args: [bookingId, "msg-001"]);
```

📡 Everyone in the room receives:

```
{
  "messageId": "msg-001",
  "userId": "riya-id",
  "readAt": "2026-06-01T15:01:35Z"
}
```

→ Aarav's grey single-tick becomes **double blue tick ✓✓**.

🕒 3:10 PM — Reactions & Edits

Add a reaction ❤️

```
await hubConnection.invoke('AddReaction', args: [bookingId, "msg-002", "❤️"]);
```

Everyone receives `ReactionAdded` event → render heart under that message bubble.

Edit a message

```
await hubConnection.invoke('EditMessage', args: [
  bookingId,
  "msg-001",
  "Hi doctor, I've been feeling anxious lately",
]);
```

Everyone receives `MessageEdited` with full updated `ChatMessage` + `isEdited: true`. Show "edited" label below the bubble.

3:13 PM — Network Drops Briefly

The `signalr_netcore` package with `.withAutomaticReconnect()` handles reconnect automatically. After reconnecting, **your Flutter code must re-call `JoinChat`**:

```
hubConnection.onreconnected(({connectionId}) async {
  await hubConnection.invoke('JoinChat', args: [bookingId]);
});
```

Server detects Aarav already has `FirstJoinedAt` set → **no system message** (silent reconnect). Riya doesn't see any "joined" notification.

3:30 PM Sharp — Window Closes

 **Important:** `chatClosesAt = scheduledEndAt` — the chat dies **at the exact second** the booking ends. No 5-minute buffer, no wrap-up time. **3:30:00 → input bar locked.**

Your local timer (from `chatClosesAt`) fires at 3:30 PM sharp → disable composer, show banner.

If Aarav still tries to send (e.g., he had a half-typed message at 3:29:58):

 **Server fires:**

```
{
  "code": "SESSION_ENDED",
  "message": "Your session has ended. Please book again to continue chatting."
}
```

Flutter handling:

```
hubConnection.on('ChatAccessDenied', (args) {
  final data = args![0];
  switch (data['code']) {
    case 'SESSION_ENDED':
      showBookAgainBanner();
      disableComposer();
      break;
    case 'SESSION_NOT_STARTED':
      showCountdown(data['chatOpensAt']);
      break;
    case 'BOOKING_CANCELLED':
      showCancelledBanner();
      break;
  }
});
```

REST `POST /chat/send` equivalent returns **HTTP 403** with the same body.

4:00 PM — Aarav Re-opens to Re-read

1. `GET /chat/access` → `canRead: true, canWrite: false, code: "SESSION_ENDED"`
2. `GET /chat/history` → returns **all messages** (still readable!)
3. Render full history, input bar locked, "Book Again" CTA at bottom
4. **Do not** connect SignalR — read-only mode

The `MessageType` field is just a string on the wire — there's no strict enum on the backend. Currently the system understands these values:

Type	Sent by	When to use	Special behavior
Text	Client	Default — normal chat text	None — just stored & broadcast
Image	Client	Photo upload (jpg, png, gif, webp)	attachment field required
File	Client	Any non-image file (PDF, doc, etc.)	attachment field required
Audio	Client	Voice note recording	attachment field required
System	Server only	Welcome, "X has joined", future "session ended" notices	Immutable — can't be edited, deleted, or reacted to

Complete API Reference

REST APIs

Method	Endpoint	When Flutter Calls It
GET	/chat/access/{bookingId}	Before opening chat; poll every 30s while open
GET	/chat/history/{bookingId}?page=1&pageSize=50	On chat screen open; pagination "load older"
GET	/chat/conversation/{bookingId}	Get participants + online status + unread count
GET	/chat/search/{bookingId}?q=text	Search bar in chat header
POST	/chat/send	Fallback if SignalR unavailable
PUT	/chat/message/edit	Fallback edit
DELETE	/chat/message/{messageId}	Fallback delete

SignalR — Methods You Invoke

Method	Args	When to Call
JoinChat	bookingId	After connecting; after reconnecting
LeaveChat	bookingId	On screen dispose / navigate away
SendMessage	bookingId, content, type, idempotencyKey, attachmentJson	User sends message

EditMessage	bookingId, messageId, newContent	User edits own message
DeleteMessage	bookingId, messageId	User deletes own message
AddReaction	bookingId, messageId, emoji	User taps emoji
RemoveReaction	bookingId, messageId, emoji	User removes reaction
StartTyping	bookingId	On first keystroke
StopTyping	bookingId	After 3s no typing (debounce)
MarkRead	bookingId, messageId	When message scrolls into view
MarkConversationRead	bookingId	Bulk: when screen opens
Heartbeat	bookingId (optional)	Every 30s to stay online

SignalR — Events You Listen To

Event	Payload	What Flutter Does
MessageSent	ChatMessage DTO	Append to chat list (or replace optimistic by idempotencyKey)
MessageEdited	ChatMessage DTO	Update in place; show "edited" label
MessageDeleted	{messageId, deletedAt}	Replace content with "[Message deleted]"
MessageDelivered	{messageId, deliveredAt}	Single grey tick

MessageRead	{messageId, userId, readAt}	Double blue tick
ReactionAdded	{messageId, userId, emoji}	Add reaction bubble under message
ReactionRemoved	{messageId, userId, emoji}	Remove reaction
TypingStarted	{userId, bookingId}	Show "X is typing..." indicator
TypingStopped	{userId, bookingId}	Hide indicator
UserOnline	{userId, bookingId}	Green dot ON
UserOffline	{userId, bookingId}	Green dot OFF
ConversationRead	{bookingId, userId, readAt}	Mark all your sent as read
ChatWindow	{chatOpensAt, chatClosesAt, ...}	Save closesAt; start local auto-disable timer
ChatAccessDenied	{code, message, ...}	Show banner; disable composer
HeartbeatAck	{timestamp}	Connection alive — no UI action
Error	{code, message}	Show toast

Error Codes Reference

Code	Meaning	Flutter Action
------	---------	----------------

OK	Chat allowed	Enable everything
SESSION_NOT_STARTED	Before chatOpensAt	Show countdown waiting room
SESSION_ENDED	At or after scheduledEndAt	Show "Book Again" CTA, disable composer
BOOKING_CANCELLED	Booking was cancelled	Show cancelled banner, permanent lock
NOT_PARTICIPANT	User not on this booking	Navigate back / show error
BOOKING_NOT_FOUND	Invalid bookingId	Show error, navigate back
RATE_LIMIT	Too many messages (max 30/min)	Toast: "Slow down..." — let user retry

Flutter Architecture Tips

1. ChatService singleton

Manage `HubConnection` as a singleton (provider/riverpod/get_it). Don't recreate it on every screen mount.

2. Use clockDrift

Compute drift once from `serverTimeUtc` on access check. All countdowns/timers use `DateTime.now() - drift` instead of raw device time.

3. IdempotencyKey is non-negotiable

Generate with `Uuid().v4()` before sending. Server rejects duplicates — Flutter retries on timeout become safe.

4. Optimistic UI

Add the message locally with a "sending" state *before* the SignalR invoke completes. Match it back when `MessageSent` arrives via `idempotencyKey`.

5. AppLifecycleState

On `paused` → call `LeaveChat` and stop heartbeat. On `resumed` → reconnect + `JoinChat` again. Re-check access in case window closed while backgrounded.

6. The 30-second poll

While the chat screen is open, run a periodic `GET /chat/access` every 30s. Cheap insurance — catches edge cases like admin-cancelled bookings.

7. Local auto-lock at `chatClosesAt`

Since the window is **strict** (closes exactly at `scheduledEndAt`), your local `Timer` must be precise. Use `Timer(chatClosesAt - now, () => disableComposer())`. This way the UI locks even before the user attempts a send.

One-Line Mental Model

REST = the menu (check access, load history, search)
SignalR = the live conversation (join, send, type, react)
Access check = the bouncer — decides if the door is open *right now*

Generated for Humora Chat — .NET Backend + Flutter Mobile
Window: 5 min before scheduled start → exactly at scheduled end (no buffer)